

Средства симуляции ЦП и ОС

Лекция №12 – 13



Державин Андрей
Шурыгин Антон

Тестирование

Зачем?



works on my machine



Поиск ошибок

- Программа работает верно?
- Какие ограничения работы?
- Что по перфу?

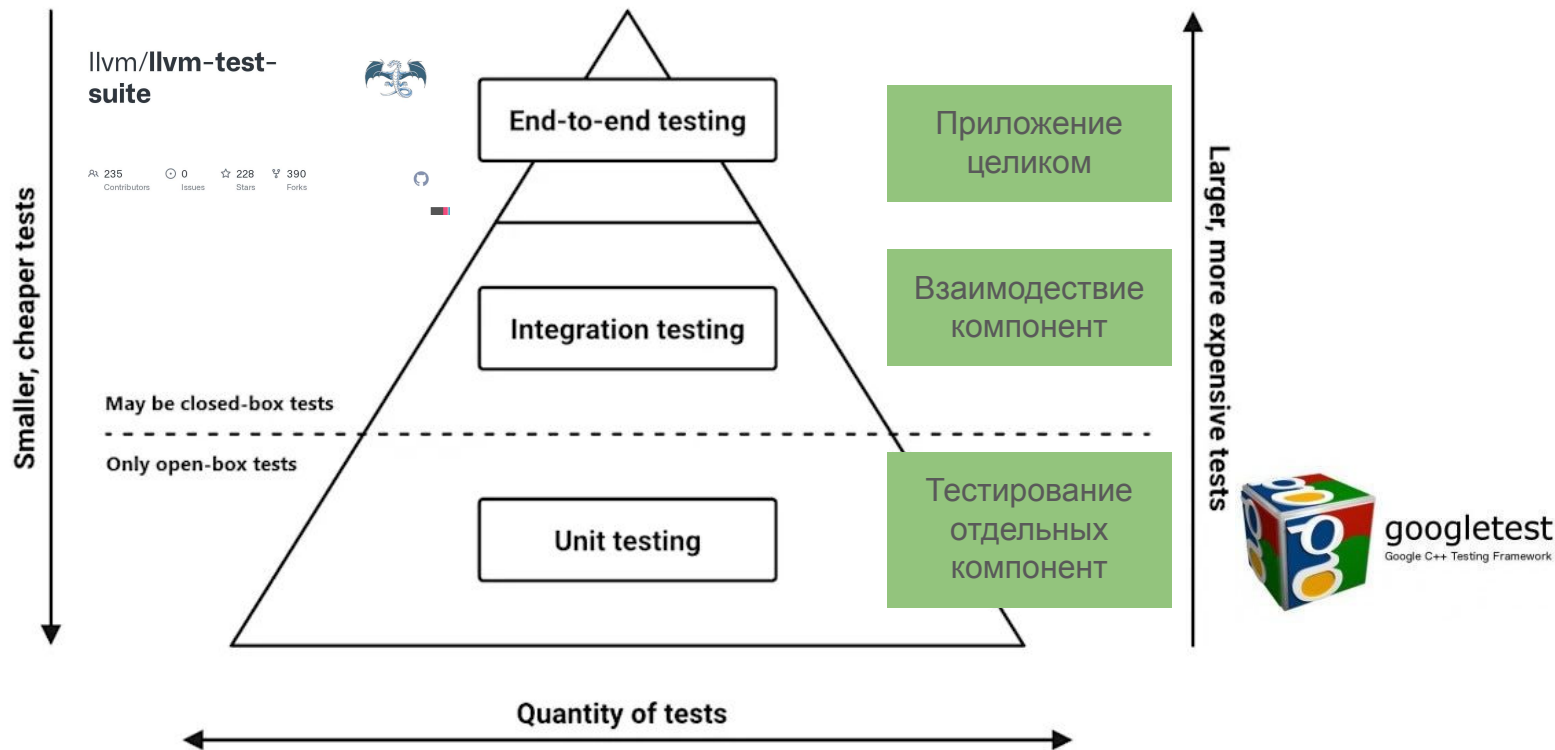
Обнаружение регрессий

- Я не сломал существующий код?
- А может он уже был сломан...

Воспроизводимость

- хз, у меня работает...
- есть ли проблемы при сборке с нуля?
- везде ли собирается?

А что это?



Unit testing. AAA

- Unit Test подразделяют на 3 этапа
- Arrange
 - Задание начальных условий
- Act
 - Исполнение тестируемого кода
- Assert
 - Проверка результатов

Больше про gtest [по ссылке](#)

```
int add(int a, int b) { return a + b; }
```

```
TEST(AddTest, PositiveNumbers) {  
    // Arrange  
    int x = 2;  
    int y = 3;  
    int expected = 5;  
  
    // Act  
    int result = add(x, y);  
  
    // Assert  
    ASSERT_EQ(result, expected);  
}
```

Автоматизированное тестирование

Автоматизированное тестирование

- Различают различные подходы для автоматизированного (и не очень) тестирования
 - Модульное тестирование
 - Интеграционное тестирование
 - Регрессионное тестирование
 - Тестирование на основе формальных моделей
 - Символическое исполнение
 - Формальная верификация
 - Мутационное тестирование
 - Статический анализ
 - ...

(Автоматическое) тестирование

- Наиболее эффективны для тестирования моделей ЦПУ:
 1. Co-simulation[\[1\]](#)
 2. Formal Verification[\[4\]](#)
 3. Differential, Fuzz Testing[\[2, 4\]](#)
 4. Symbolic Execution[\[3, 4\]](#)

Formal
(критические пути)

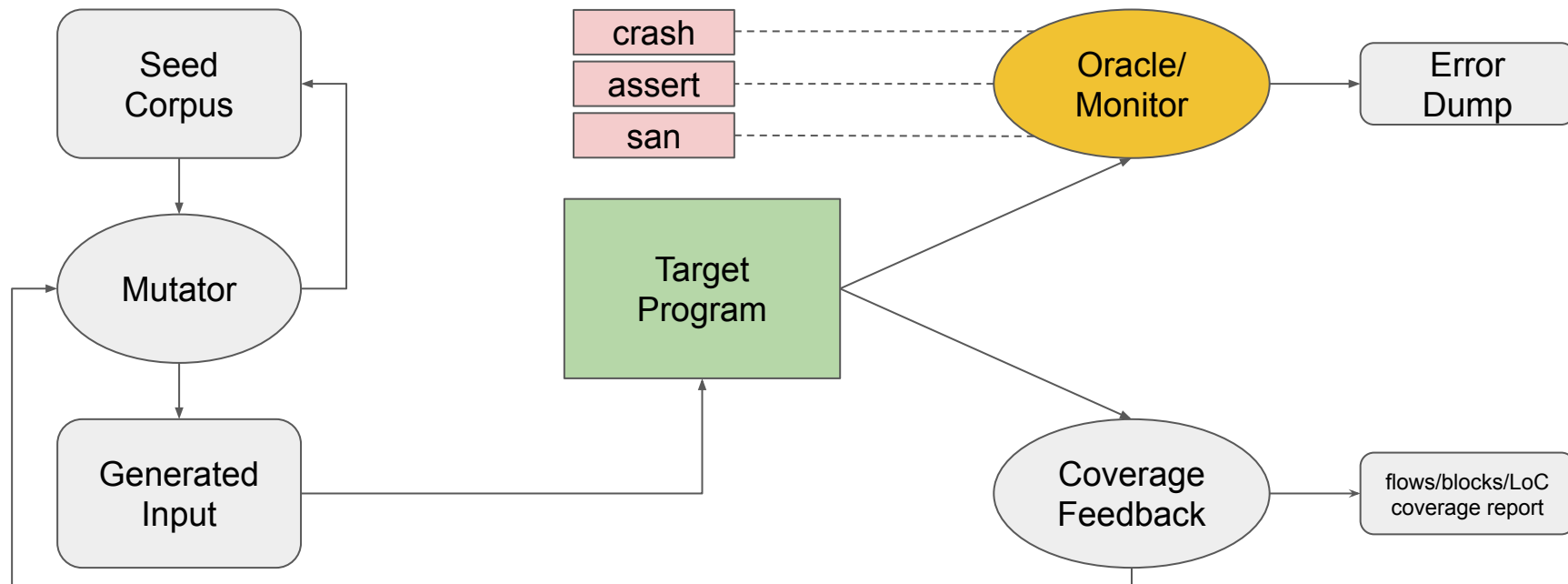
Co-simulation
(функциональность)

Coverage Guided
(анализ состояний)

Fuzzing

Fuzzing

- Обычно выделяют следующие ключевые компоненты



Обобщенная архитектура рабочего процесса fuzz-тестирования

Типы fuzz-тестирования

- Blacbox Fuzzing
 - Тестирование не имеет доступа к исходному коду
 - Мутирование входных данных

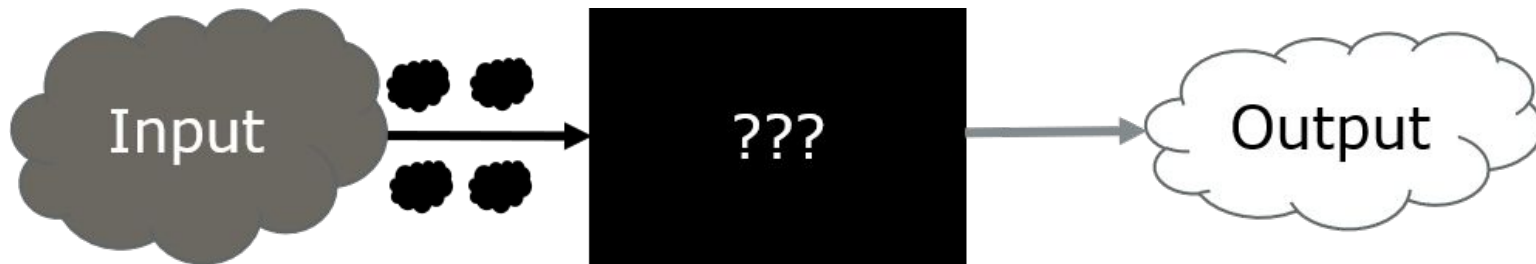


Схема "черного ящика" fuzz-тестирования

Типы fuzz-тестирования

- Whitebox Fuzzing

- Тестирование обладает полным знанием о программе
- Обычно использует символическое исполнение + SMT-решатель для генерации входов

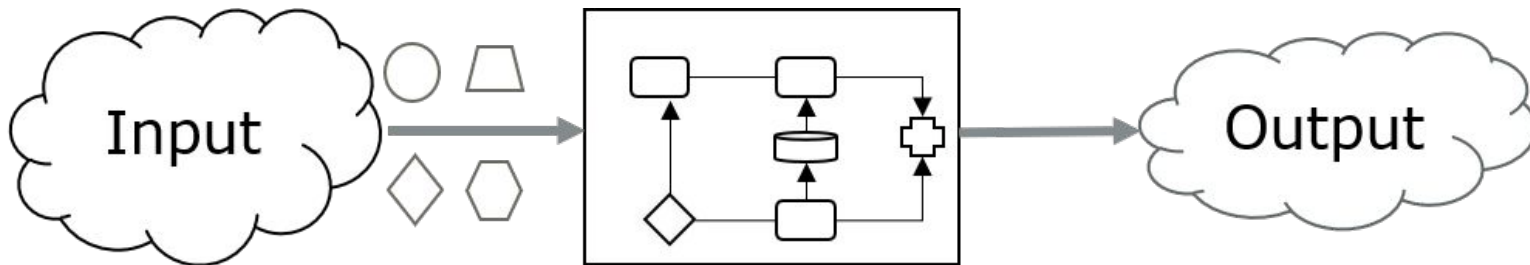


Схема "белого ящика" fuzz-тестирования

Типы fuzz-тестирования

- Greybox Fuzzing (серый ящик)
 - Тестирование использует частичную (легковесную) информацию об исполнении программы
 - Компромисс между blackbox и whitebox

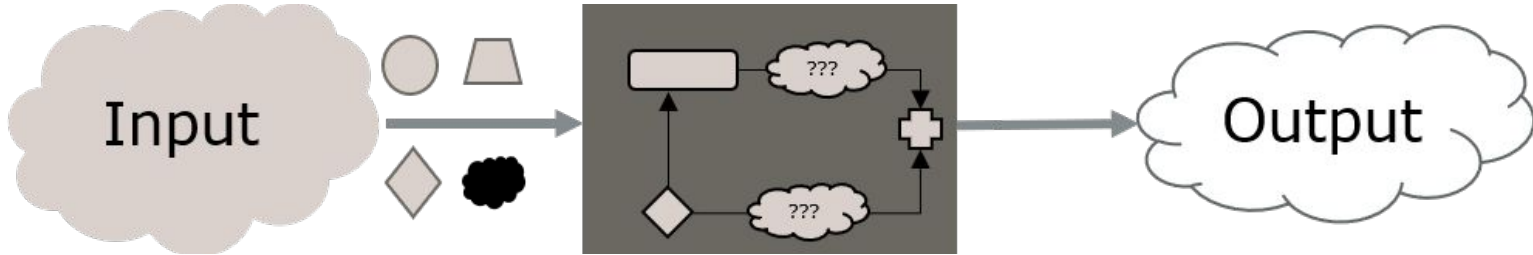
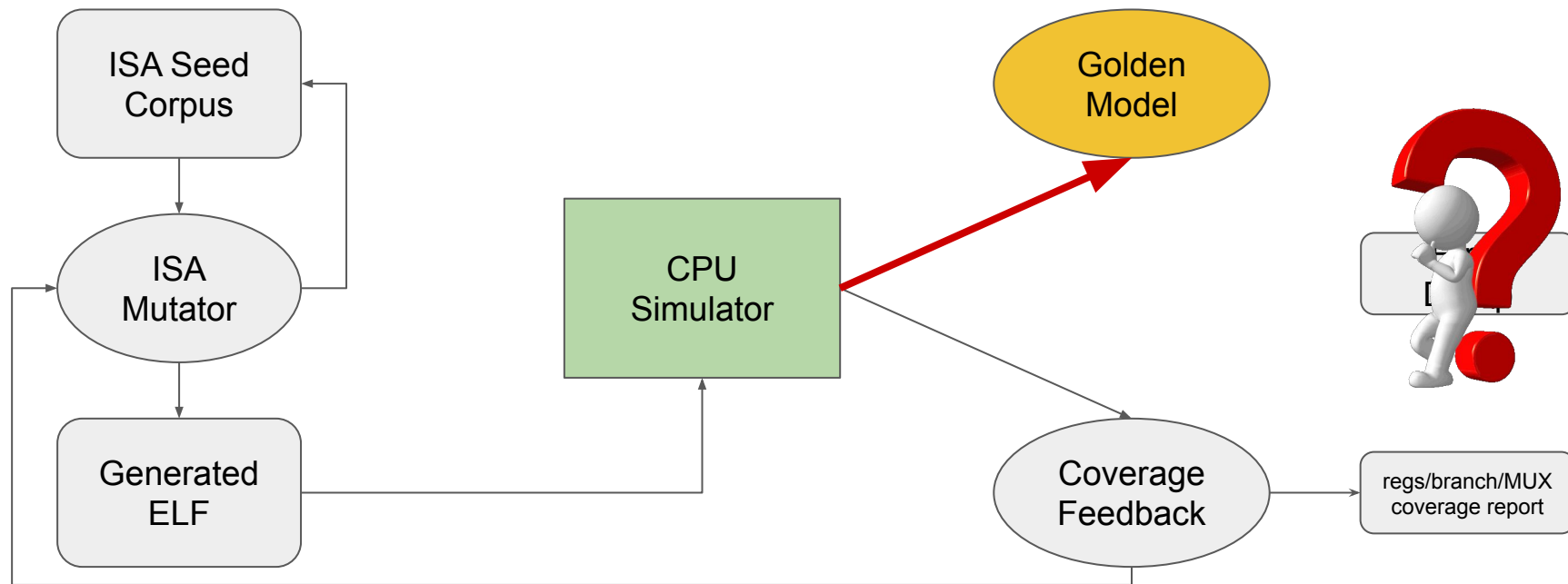


Схема "серого ящика" fuzz-тестирования

CPU Model Fuzzing

- Fuzzer для целей автоматизированного тестирования модели ЦПУ

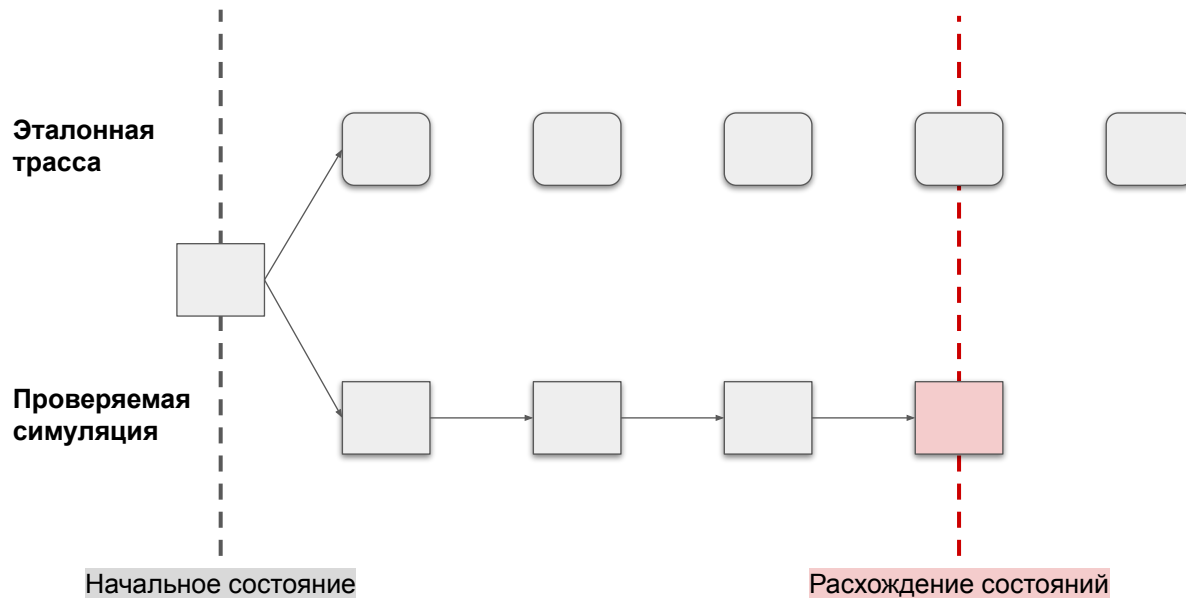


Специализация fuzzing для целей тестирования модели ЦПУ

Co-simulation

Co-simulation

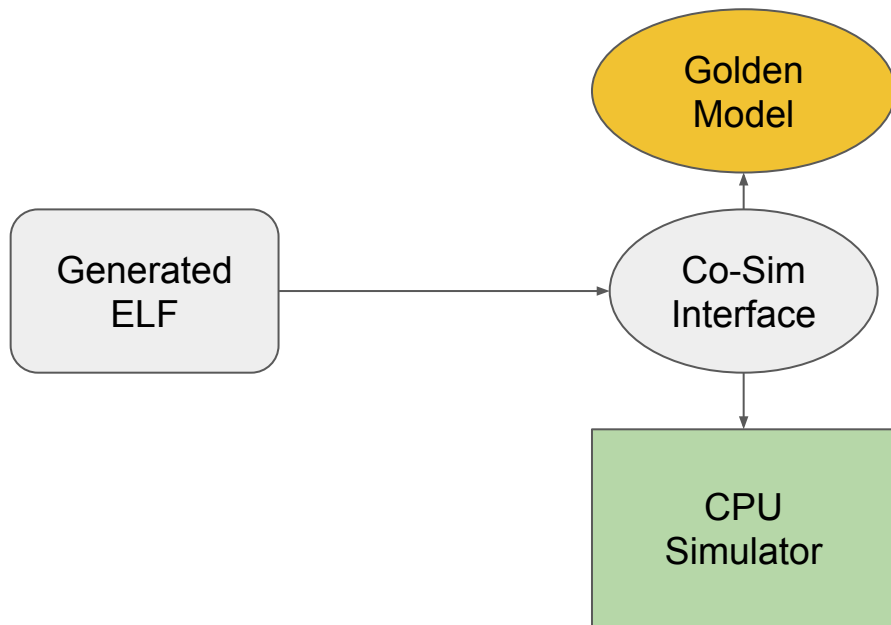
- Верификация была упомянута в лекции про симуляцию по трассам:



Верификация модели с помощью эталонной трассы на каждом шаге симуляции

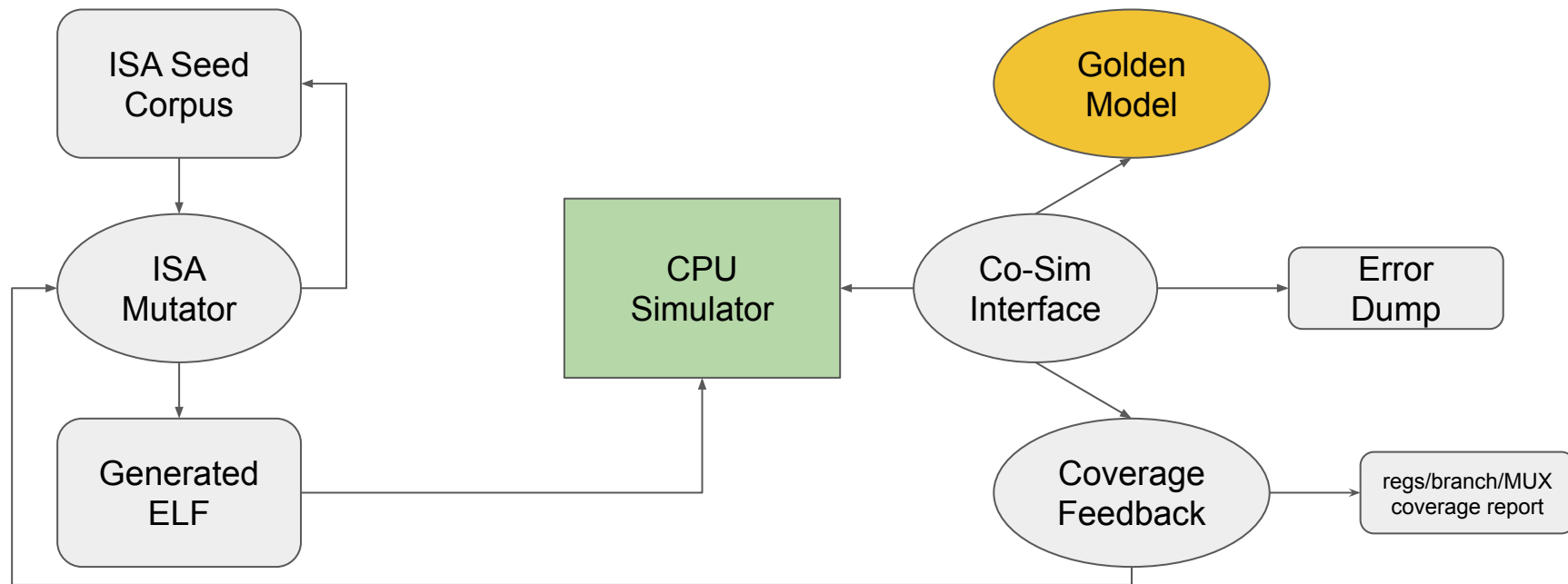
Co-simulation

- Метод параллельного исполнения двух моделей для взаимной верификации



CPU Model Fuzzing

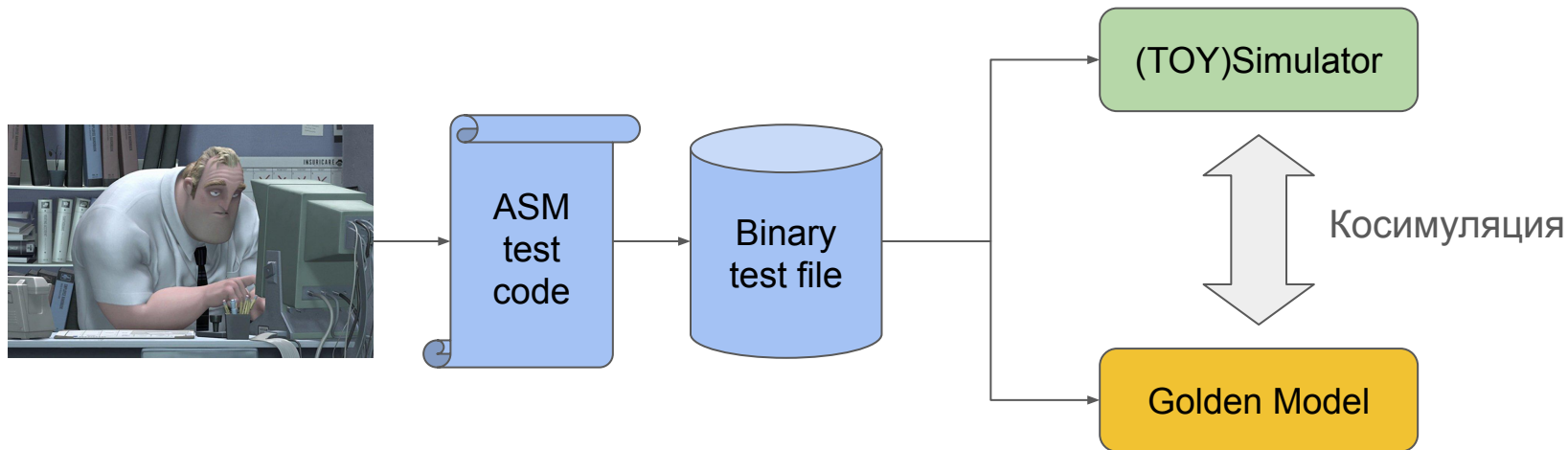
- Fuzzer для целей автоматизированного тестирования модели ЦПУ



Примеры реализации

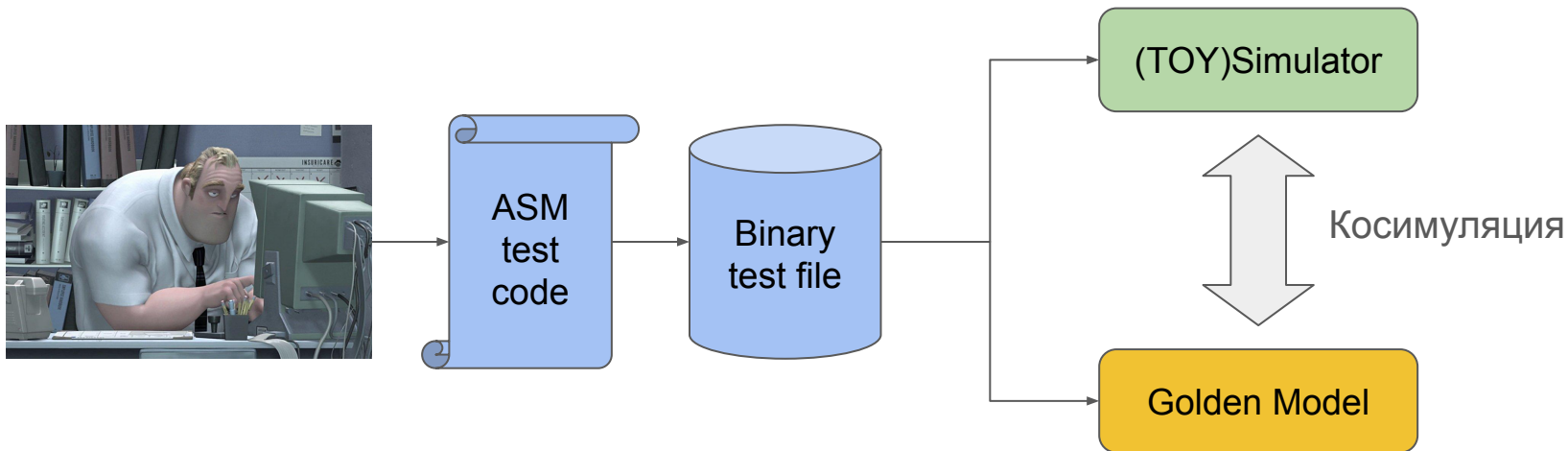
Наивный генератор

- Попробуем изобрести генератор
- Как он может работать?
- Пишем тест -> компилируем -> сверяем с golden model



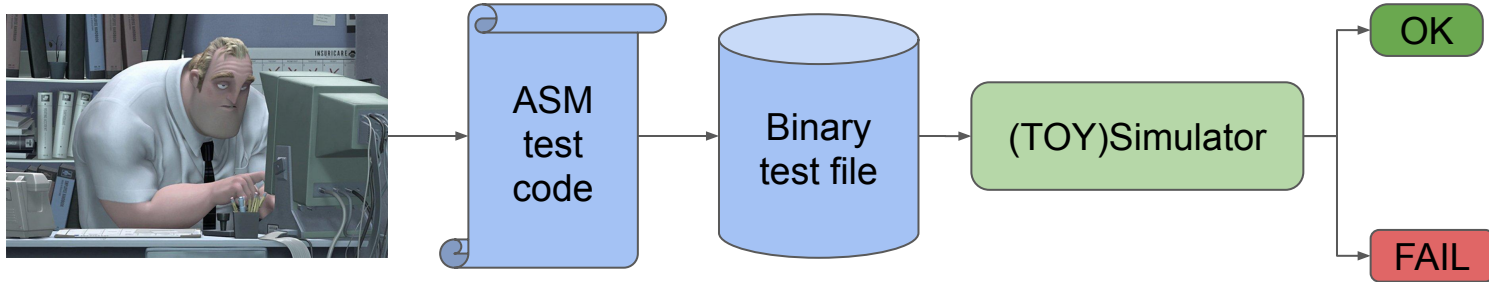
Наивный генератор

- Золотая модель не всегда доступна
- Поддержка косимуляции с Золотой моделью - не быстрое дело
- А для чего нам нужна косимуляция?
 - Сравнение с референсом



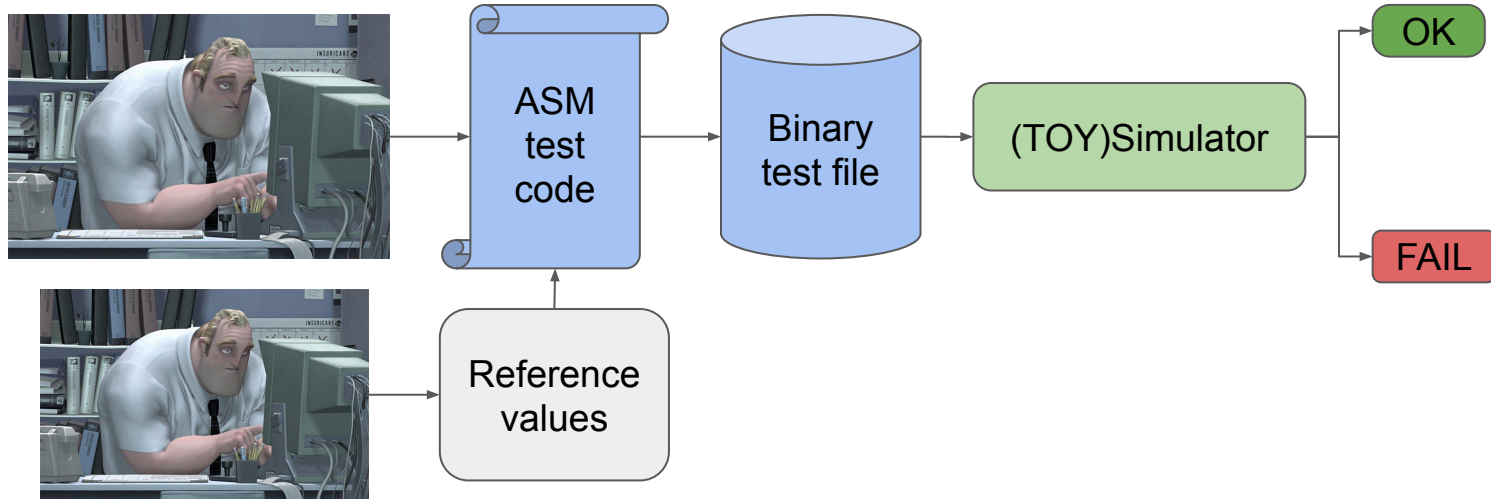
(мнее) Наивный генератор

- Сформулируем цель
- Как заставить симулятор самостоятельно проверять результаты теста?
 - Сравнивать с референсами



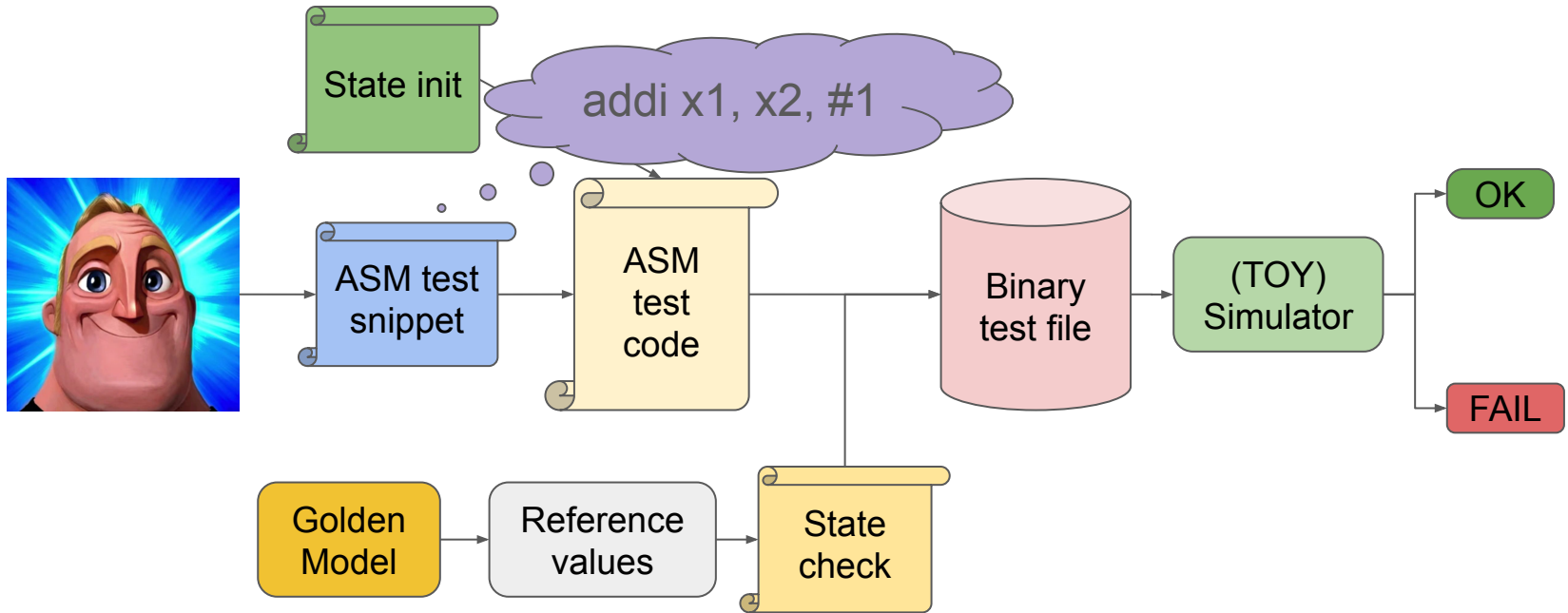
(мнее?) Наивный генератор

- Кажется мы увеличили объём работы
- И зачем-то выкинули золотую модель
- Можем ли мы автоматизировать проверку референсных значений?
 - Вообще говоря да



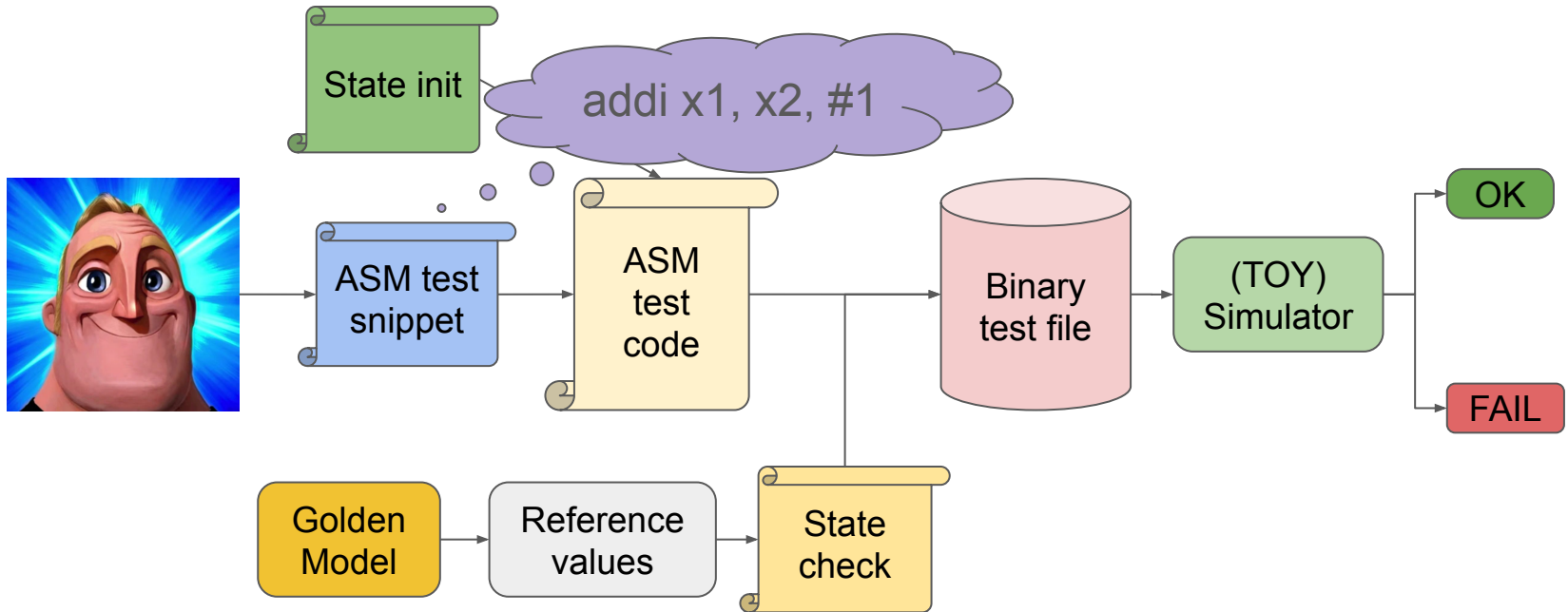
(уже не) Наивный генератор

- Попробуем описать обвязку с проверками, в которую встраивается код теста



(уже не) Наивный генератор

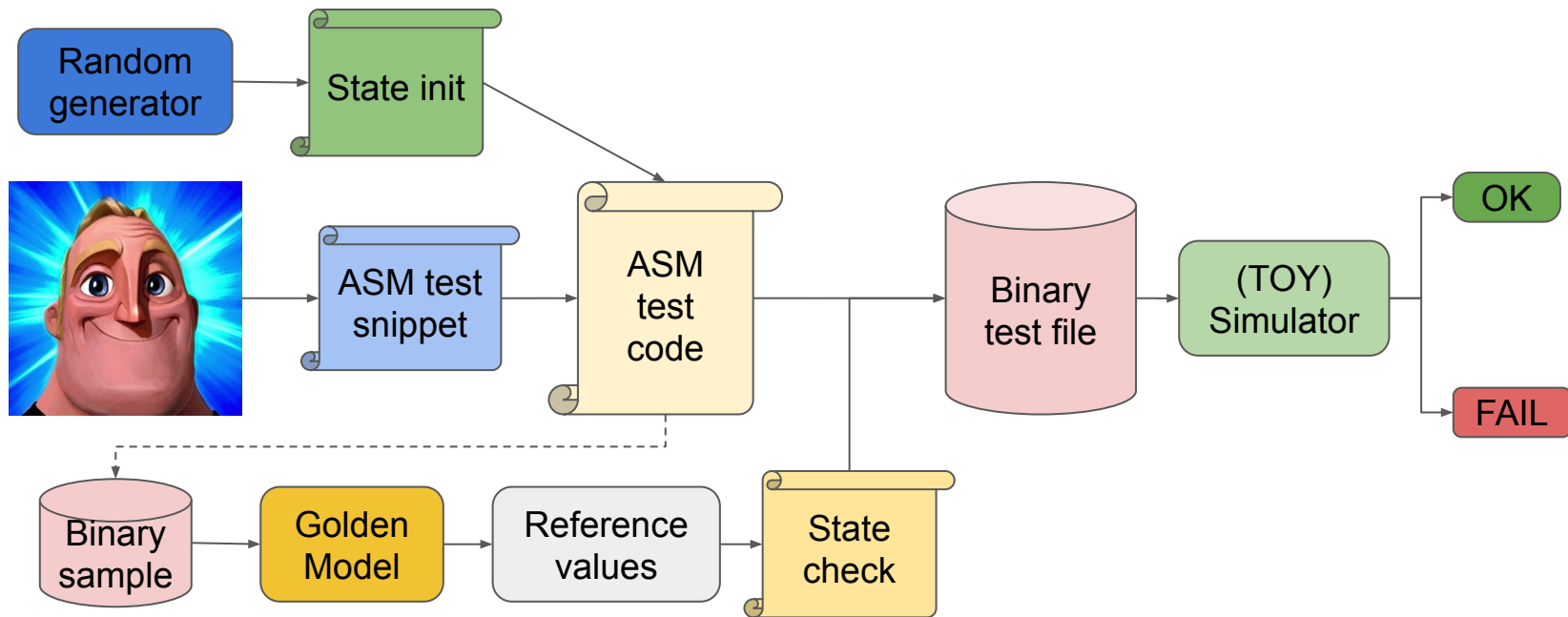
- Попробуем описать обвязку с проверками, в которую встраивается код теста



Random Snippet Testing

(уже не) Наивный генератор

- Добавим деталей

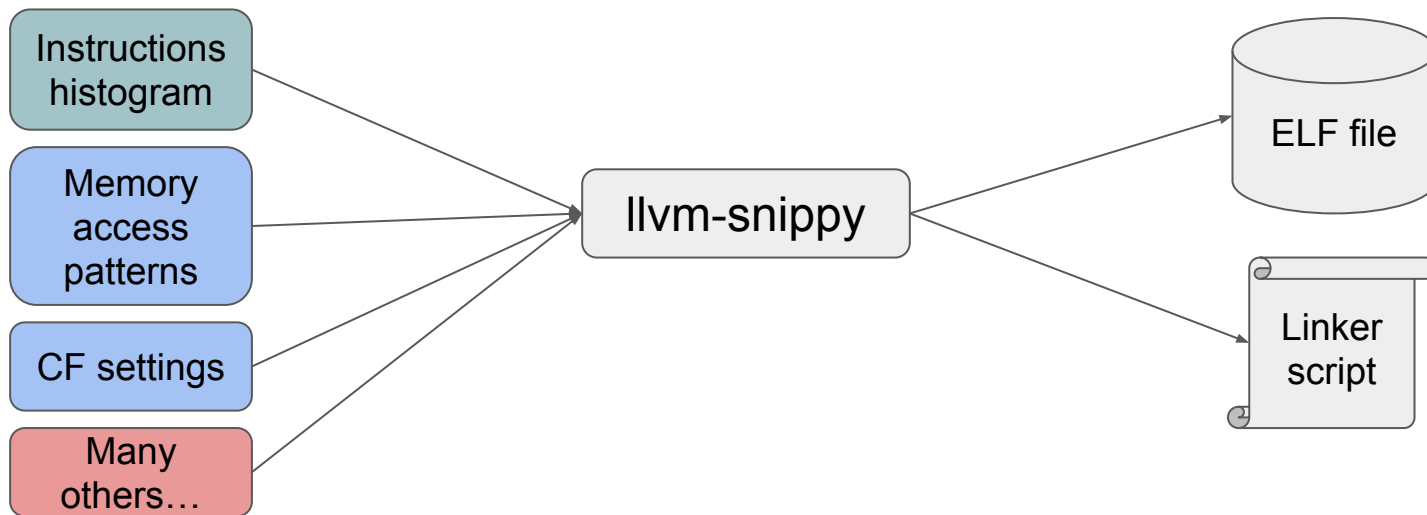


Автоматизация

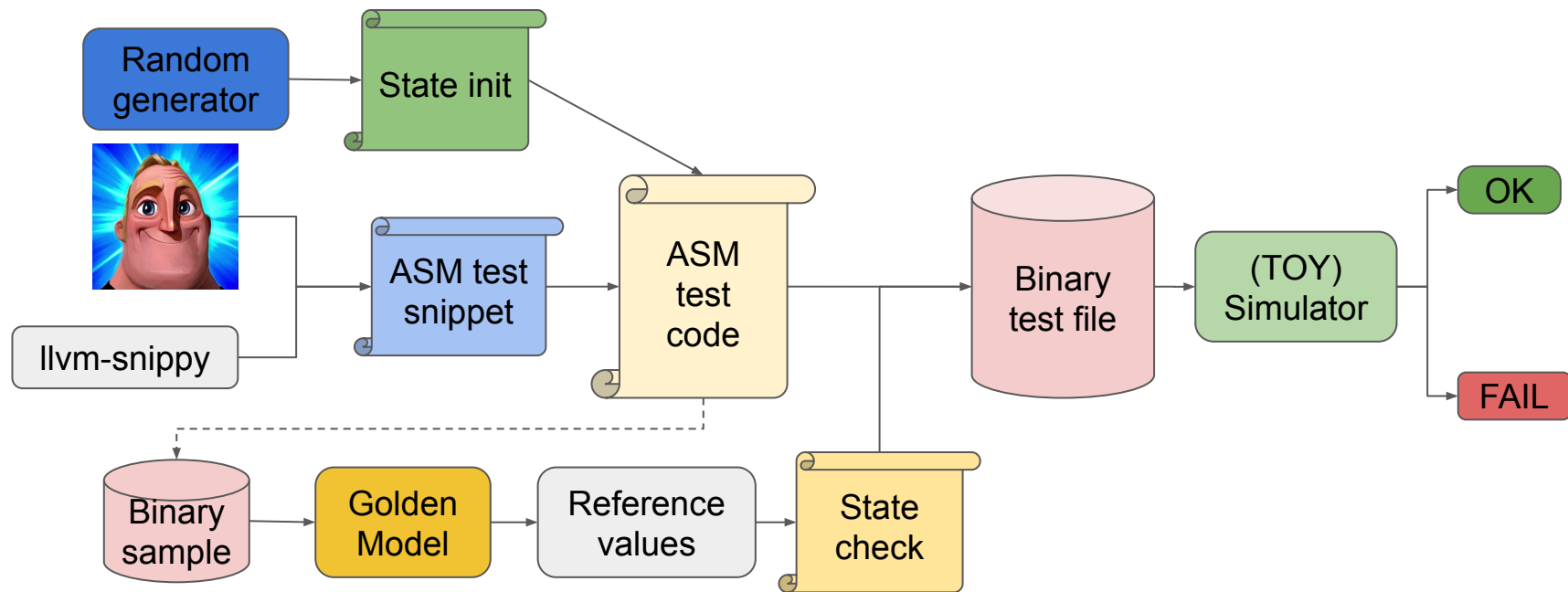
- Приведённая схема хорошо подходит для ручного тестирования
 - Краевые случаи, регрессии...
- Кажется что этого не достаточно
- Нужна программа, которая будет генерировать всевозможные снippets
- Instruction Sequence Generator (ISG)
- Один из свежих примеров - llvm-snippy

llvm-snippy

- Изначально разработан компанией Syntacore
- В данный момент ведется активная разработка
- Информацию об инструкциях берет из описаний LLVM
- Поддержан RISC-V в качестве бэкенда



(совсем не) Наивный генератор

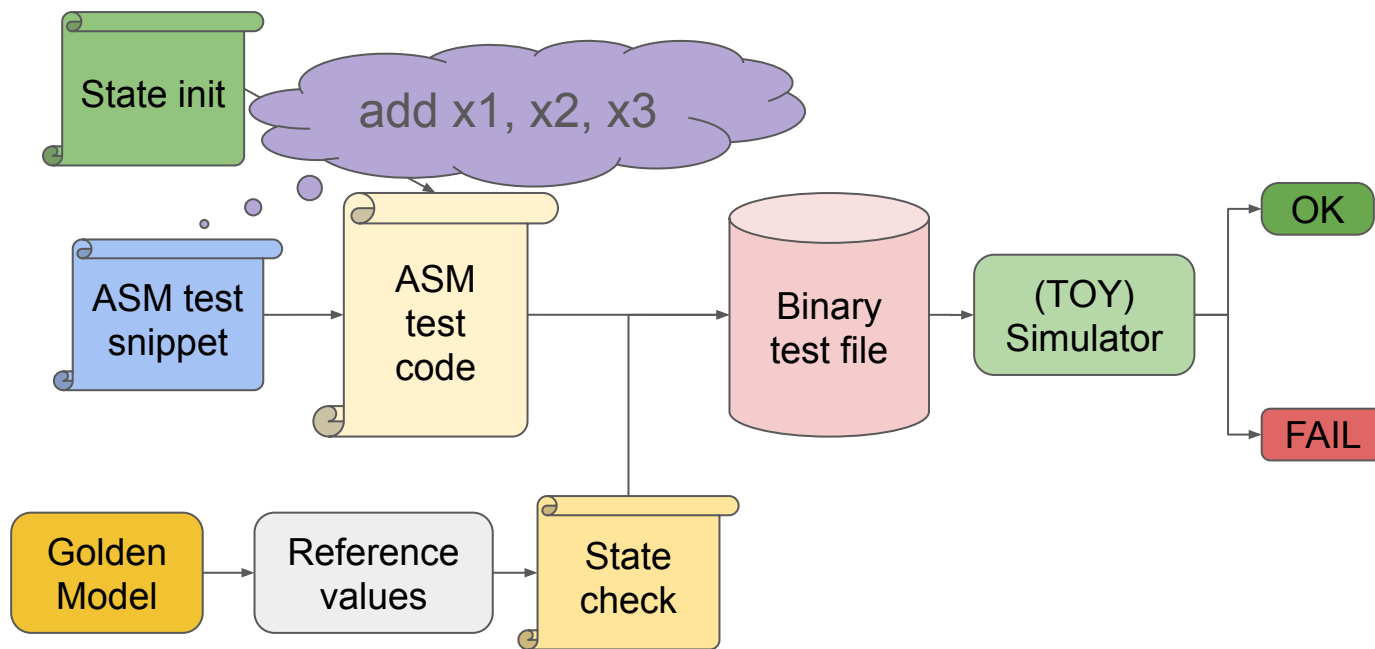


ДЗ-1

- Реализуйте собственный генератор тестов
 - Level 1: ручное задание test case, обвязка с рандомными значениями и проверкой генерится
 - **Level 2: добавьте llvm-snippy**

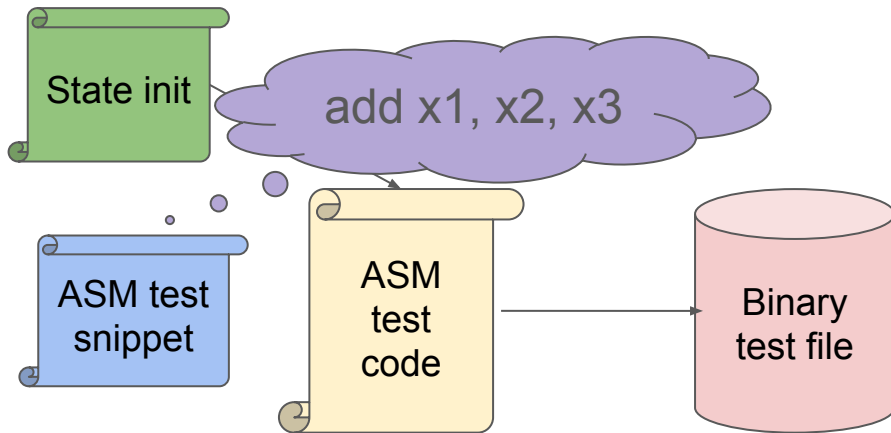
Символическое исполнение

Мотивация



Мотивация

- Для инструкции `add x1, x2, x3` руками мы можем покрыть очень малую долю:
 - Перебор всех значений для 32-битных значений затруднителен: $2^{32} * 2^{32}$ комбинаций
 - В идеале – всё и сразу
- Хочется подход, который позволит рассуждать о всех возможных значениях одновременно



Стандартное исполнение

x	y	t
4	4	0
4	4	4

```
1 void foo(int x, int y) {  
2     int t = 0;  
3     if (x > y)  
4         t = x;  
5     else  
6         t = y;  
7  
8     if (t < x)  
9         abort();  
10 }
```

Стандартное исполнение

x	y	t
2	1	0
2	1	2

```
1 void foo(int x, int y) {  
2     int t = 0;  
3     if (x > y)  
4         t = x;  
5     else  
6         t = y;  
7  
8     if (t < x)  
9         abort();  
10 }
```

Символическое исполнение

x	y	t
ξ	ψ	0
ξ	ψ	τ_0

$$\tau_0 = \begin{cases} (\xi > \psi) \Rightarrow \xi \Rightarrow \boxed{\begin{matrix} \tau_0 < \xi \\ (\xi > \psi) \Rightarrow \tau_0 = \xi \end{matrix}} \\ (\xi \leq \psi) \Rightarrow \psi \Rightarrow \boxed{\begin{matrix} \tau_0 < \xi \\ (\xi \leq \psi) \Rightarrow \tau_0 = \psi \end{matrix}} \end{cases}$$

$\Rightarrow$

```

1  void foo(int x, int y) {
2      int t = 0;
3      if (x > y)
4          t = x;
5      else
6          t = y;
7
8      if (t < x)
9          abort();
10 }

```

abort(); не достигим

Возможно ли $t < x$?

Символическое исполнение

x	y	t
ξ	ψ	0
ξ	ψ	τ_0

$$\tau_0 = \begin{cases} (\xi > \psi) \Rightarrow \xi - 1 \Rightarrow \tau_0 < \xi \\ (\xi > \psi) \Rightarrow \tau_0 = \xi - 1 \\ (\xi \leq \psi) \Rightarrow \psi \Rightarrow \tau_0 < \xi \\ (\xi \leq \psi) \Rightarrow \tau_0 = \psi \end{cases}$$

Возможно ли $t < x$?

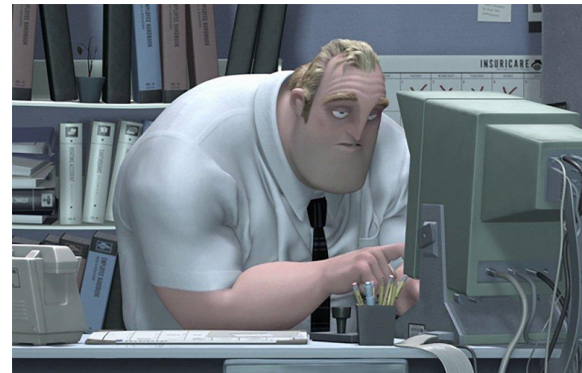
```

1 void foo(int x, int y) {
2     int t = 0;
3     if (x > y)
4         t = x - 1;
5     else
6         t = y;
7
8     if (t < x)
9         abort();
10 }
```

$\Rightarrow$ abort(); достигим при любых $\xi > \psi$

От частного – к общему

- Продемонстрировали как “на пальцах” символически исполнить программу
 - Есть запрос тестировать бОльшие программы
- А можно ли как-то автоматизировать процесс?
 - Каким образом создавать формулы?
 - Как решать эти формулы?

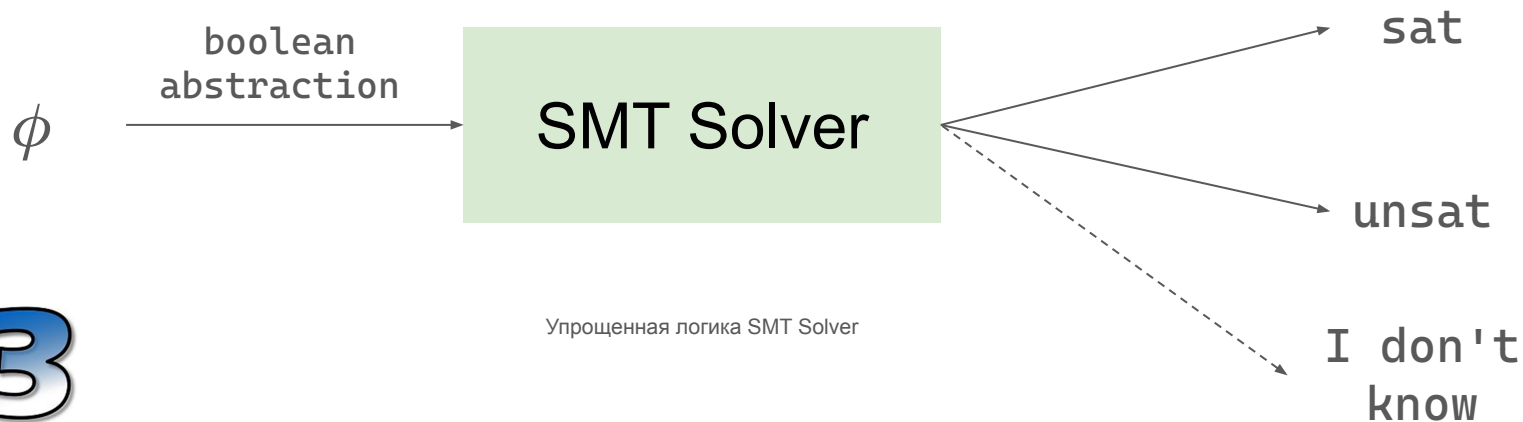


SMT

angr



- SMT (англ. Satisfiability Modulo Theories) – решатели задач выполнимости логических формул с учётом теорий
- Подробная архитектура SMT – [\[2\]](#)
- Верификации моделей, формальная верификация, статический анализ...



Z3

Z3

- Z3 — это один из самых популярных SMT-решателей, разработанный Microsoft Research.
- Поддерживает множество теорий:
 - битовые векторы
 - целые и вещественные числа
 - массивы
 - кванторы
- API для C++, Python, Java, OCaml и пр.

```
#include <iostream>
#include <z3++.h>

int main() {
    z3::context ctx;
    z3::expr x = ctx.bv_const("x", 32);
    z3::expr y = ctx.bv_const("y", 32);

    z3::expr t = z3::ite(x > y, x, y);
    z3::expr condition = (t < x);

    z3::solver s(ctx);
    s.add(condition);

    if (s.check() == z3::sat) {
        std::cout << "abort() is possible\n";
        z3::model m = s.get_model();
        std::cout << "x = " << m.eval(x)
                  << ", y = " << m.eval(y) << "\n";
    } else {
        std::cout << "abort isn't possible\n";
    }
    return 0;
}
```

Символическое исполнение
`void foo(int x, int y)`

Z3

```
$ cat symb_exec.py
```

```
import z3
```

```
x = z3.BitVec('x', 32)
```

```
y = z3.BitVec('y', 32)
```

```
s = z3.Solver()
```

```
s.add((x ^ 0x55) + (3 - (y * 12)) == 0x30)
```

```
s.check()
```

```
print(s.model())
```

```
$ python3 symb_exec.py
```

```
[y = 5, x = 60]
```

Символическое исполнение на python3 и Z3

Вспоминаем про симуляцию

```
Instruction(:add) {  
    encoding *format_r(0b0110011, 0b000, 0b00000000)  
    asm { "add {rd}, {rs1}, {rs2}" }  
    code { rd[] = rs1.u + rs2.u }  
}
```

Описание инструкции [ADD в Protea](#)

Вспоминаем про симуляцию

- В нашей DSL в древовидной форме в промежуточном представлении (IR) расписана семантика инструкции
- Идея: автоматически транслировать узлы IR в Z3 выражения:
 - Проверка свойств и инвариантов
 - Проверка эквивалентности с золотой моделью
- Какие есть очевидные сложности?

```
- :name: :add
  :oprnds:
- :name: :_tmp11
  :type: :u32
  :regset:
- :name: :_tmp7
  :type: :u32
  :regset:
- :name: :_tmp9
  :type: :u32
  :regset:
  :attrs:
- :name: :writeReg
  :oprnds:
- :name: :rd
  :type: :r32
  :regset: :XRegs
- :name: :_tmp11
  :type: :u32
  :regset:
  :attrs:
```

Покрытие кода

Всё ли тестируем?

- Написали 1000 тестов. Все проходят
- Все работает?
 - Не обязательно
- Часть кода может **не исполняться**
 - Там могут быть баги
- Code Coverage показывает, какая часть программы была исполнена во время тестов

Not instrumented →

Covered by tests →

Not covered by tests →

```
516 INSTITUTE_TEST_SUITE_P(NativeSequenced,  
517                          ThreadPoolWorkerPoolTest,  
518                          ::testing::Values(PoolExecuti  
519                          test::PoolType::NATIVE,  
520                          test::ExecutionMode::SEQU  
521 #endif  
522  
523 TEST(TaskSchedulerWorkerPoolTest, TestCodeCoverage) {  
524     bool flag = true;  
525     if (!flag) {  
526         int value = 10;  
527         EXPECT_EQ(10, value);  
528     }  
529     EXPECT_TRUE(flag);  
530 }
```

Пример кода

- Как будем тестировать?
- foo(2) - ОК
- Кажется, что рассмотрели не все варианты
- Попробуем ввести метрики покрытия

```
void foo(int x) {  
    printf("this is");  
    if (x < 1)  
        printf(" not ");  
    printf("a positive integer");  
}
```

Виды покрытия

- **Statement Coverage** процент выполненных строк кода
- **Branch Coverage** процент выполненных вариантов ветвления
- **Function Coverage** процент выполненных функций
- **Condition Coverage** процент выполненных сложных условий

- foo(2):
 - statement - 66%
 - branch - 50%
 - function - 100%
 - condition - 50%
- Как добить все до 100%?

```
void foo(int x) {  
    printf("this is");  
    if (x < 1)  
        printf(" not ");  
    printf("a positive integer");  
}
```


Покрыли всё?

- `foo(3), foo(-2) -> 100%`
 - Все хорошо?
- 100% покрытие совсем не означает **отсутствие ошибок**

```
void foo(int x) {  
    printf("this is");  
    if (x < 2)  
        printf(" not ");  
    printf("a positive integer");  
}
```

Покрытие. Итоги.

- Code Coverage - инструмент, а не цель. 100% покрытие не означает отсутствие проблем
- Находит неисполняемый код во время тестов
- Желательно замерять несколько видов покрытия (line + branch)

ДЗ-1

- Реализуйте собственный генератор тестов
 - Level 1: ручное задание test case, обвязка с рандомными значениями и проверкой генерится
 - Level 2: добавьте llvm-snippy
 - **Level 3*: проверьте инварианты и условия корректности для инструкций из IR.yaml с использованием SSE**

Литература

1. [Effective Processor Verification with Logic Fuzzer Enhanced Co-simulation, 2021](#)
2. [MIT Symbolic Execution Lecture](#)
3. [SMT-Solving for the First-Order Theory of the Reals](#)
4. [angr is an open-source binary analysis platform for Python](#)
5. [DIFUZZRTL: Differential Fuzz Testing to Find CPU Bugs, 2020](#)
6. [Path-Exploration Lifting: Hi-Fi Tests for Lo-Fi Emulators, 2012](#)
7. [SAGE: Whitebox Fuzzing for Security Testing, 2012](#)
8. [llvm-snippy](#)